Aula 04 – Processos e Escalonamento

Norton Trevisan Roman

21 de agosto de 2017

Processos

- Conceito central do SO;
  - Um processo é caracterizado por um programa em execução
  - Armazena todas as informações necessárias para executar um programa
- Mas existe uma diferença sutil entre processo e programa:
  - Um processo pode ser composto por vários programas, dados de entrada, dados de saída e um estado (executando, bloqueado, pronto)
  - Contém o código do programa e sua atividade atual

Processos × Programas

- Programa:
  - Um programa pode ter várias instâncias em execução (em diferentes processo)
  - Algoritmo codificado
  - Forma como o programador vê a tarefa a ser executada
- Processo:
  - Um processo é único
  - Código acompanhado de dados e momentos de execução (contexto)
  - Forma pela qual o SO vê um programa e possibilita sua execução

Processos em Primeiro Plano

- Interagem com o usuário
  - Leitura de um arquivo;
  - Iniciar um programa (linha de comando ou um duplo clique no mouse);

Processos em Segundo Plano

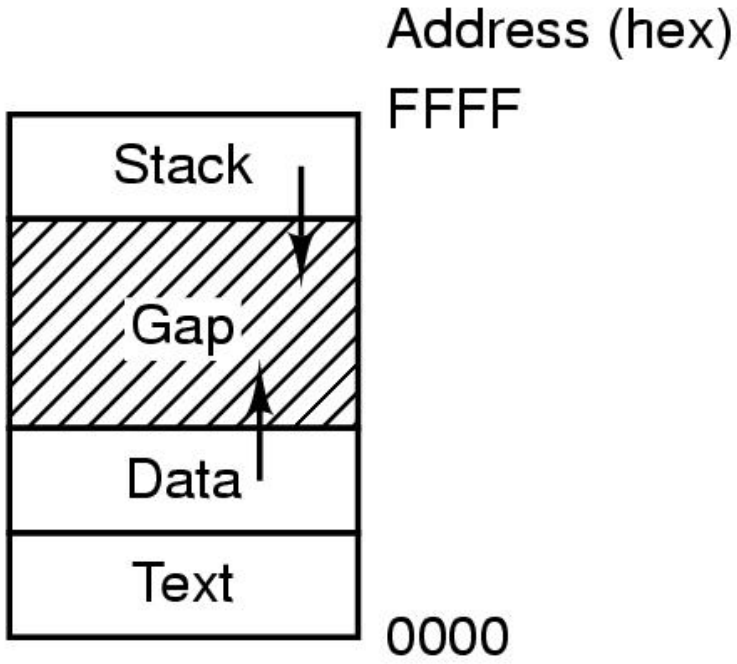
- Processos com funções específicas que independem de usuários – daemons:
  - Recepção e envio de emails;
  - Serviços de Impressão;

Processos

- Cada processo possui:
  - Programa (instruções que serão executadas);
  - Um espaço de endereçamento:
    - Lista de posições de memória, variando de 0 a um máximo, que o processo pode ler e escrever
  - Contextos de hardware: informações de registradores;
    - PC, Ponteiro da pilha, registradores de uso geral etc
  - Contextos de software: atributos;
    - Variáveis, lista de arquivos abertos, alarmes pendentes, processos relacionados etc

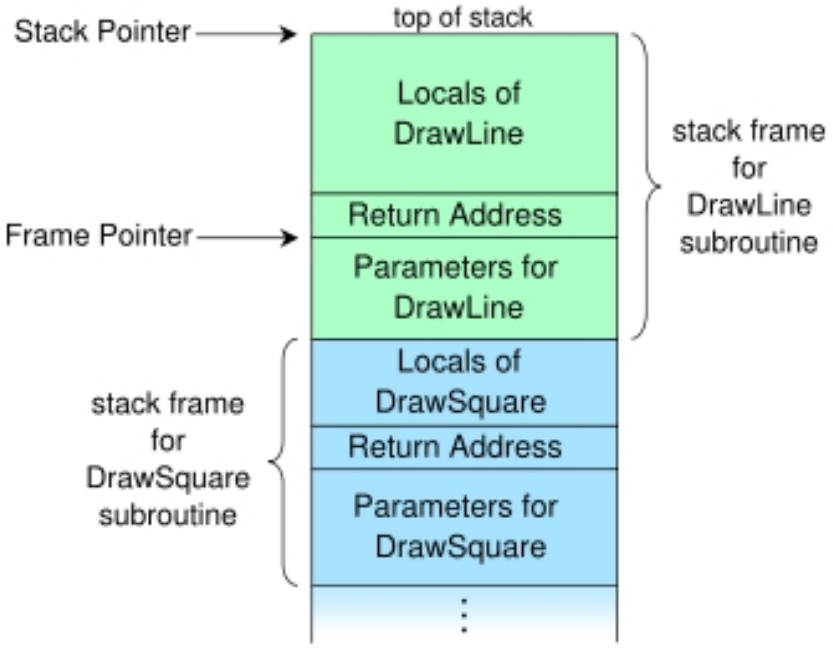
Processos – Espaço de endereçamento

- Basicamente, possui três segmentos:
  - Texto:
    - código executável do(s) programa(s);
  - Dados:
    - as variáveis;
  - Pilha de Execução:
    - Controla a execução do processo
    - Empilhando chamadas a procedimentos, seus parâmetros e variáveis locais etc



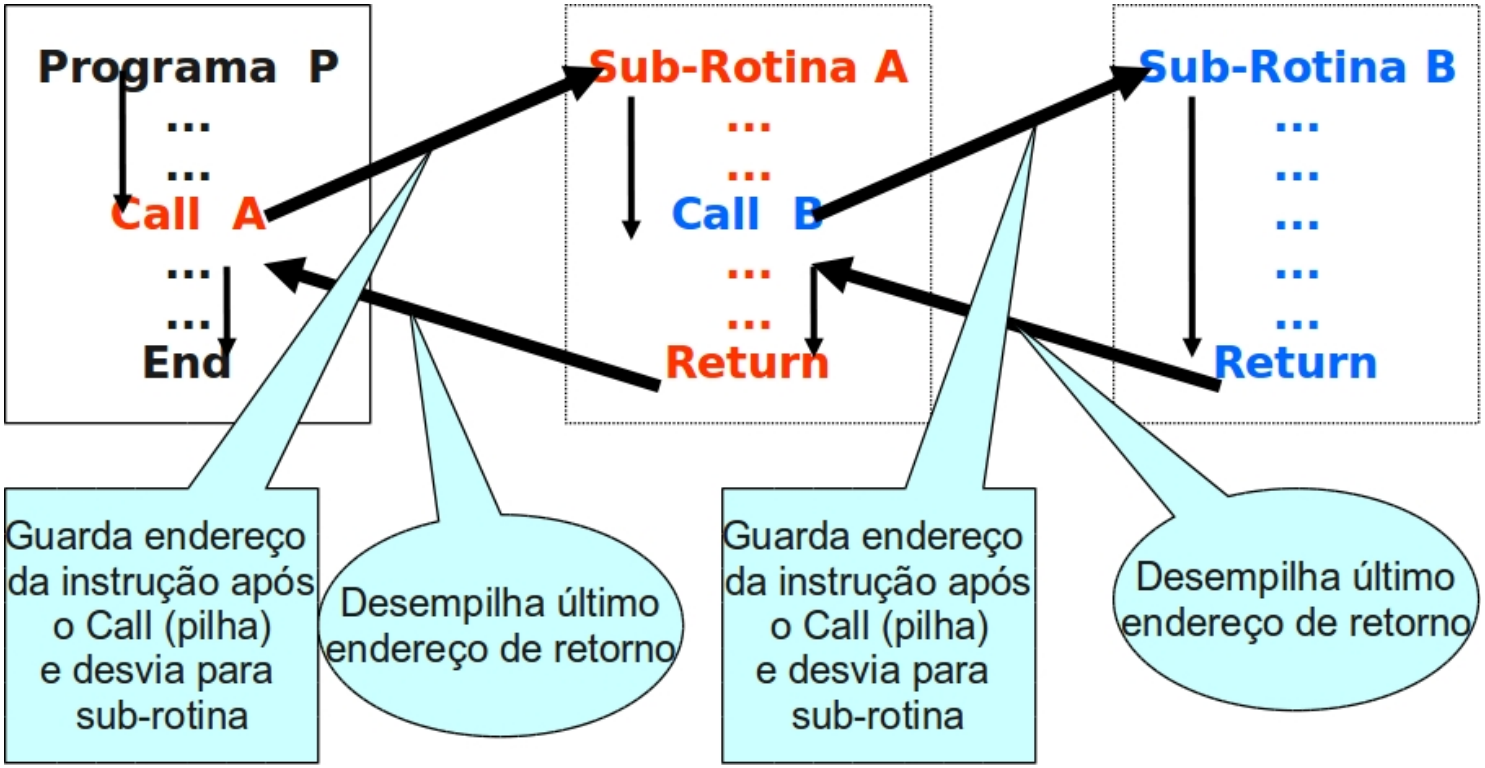
Processos – Espaço de endereçamento

- Pilha de execução:
  - Contém uma estrutura para cada rotina chamada que ainda não retornou
  - Possui:
    - Variáveis locais da rotina
    - Endereço de retorno a quem chamou
    - Parâmetros



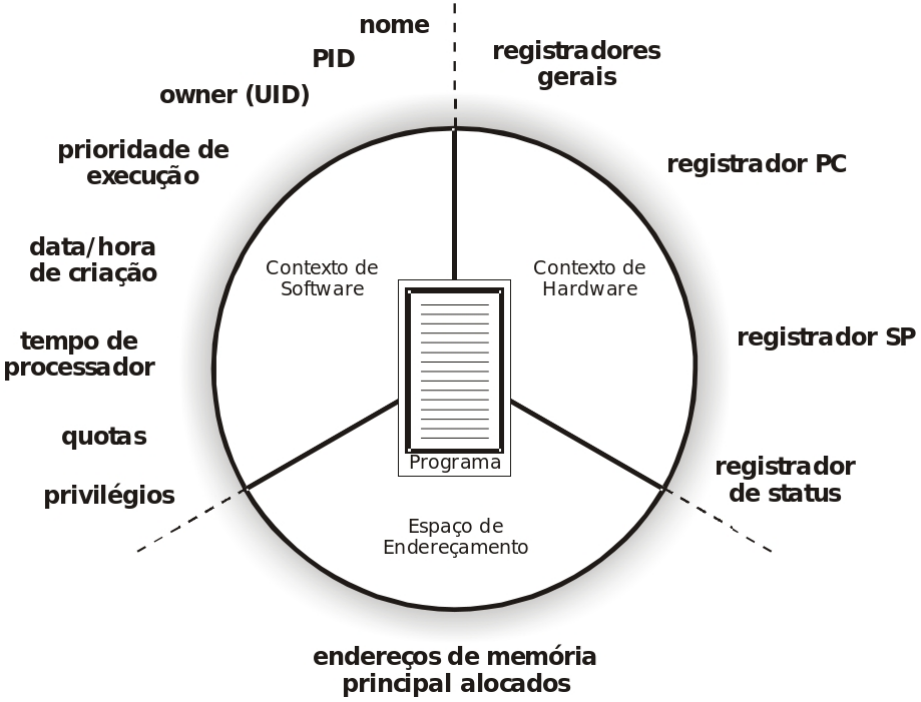
Subrotinas e a Pilha de Execução

- Funcionamento de Sub-rotinas



Processos – Contexto

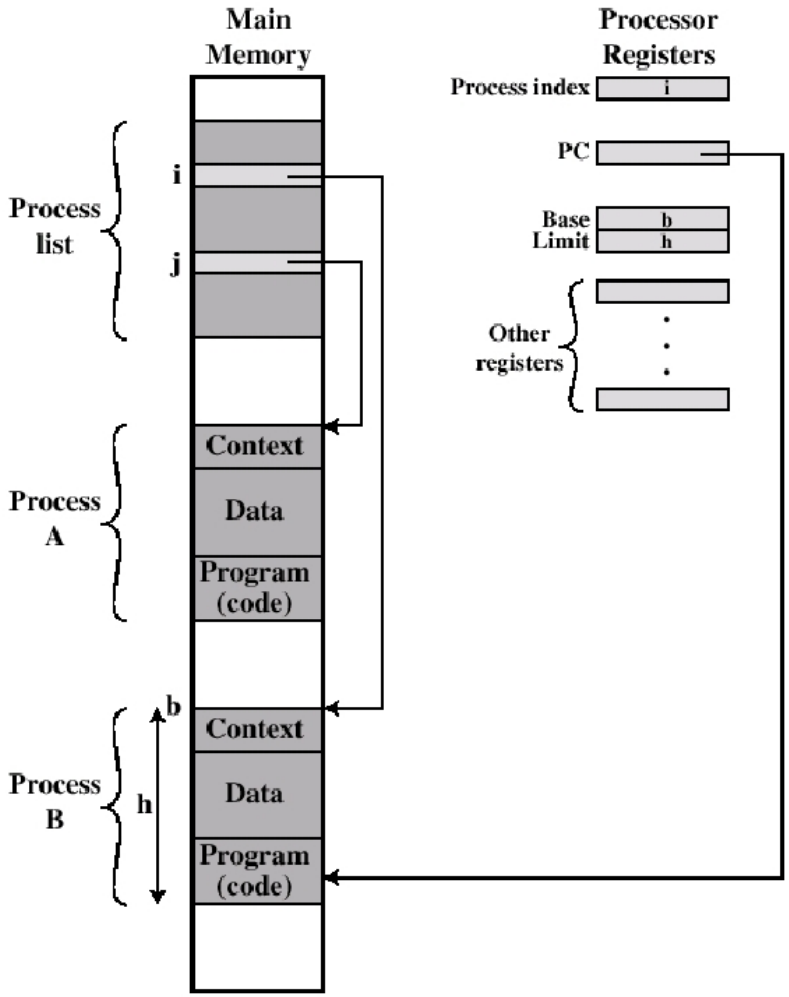
- Contém a informação de que o SO precisa para, após suspender um processo, trazê-lo de volta à execução
  - Ex: Ponteiros de arquivos abertos: posição do próximo byte a ser lido em cada arquivo



- É o estado de uma tarefa em um determinado instante

Processos – Contextos

- Dentre outras coisas, corresponde ao conteúdo dos registradores usados por um determinado processo, enquanto roda
  - Quando um processo é executado, seu contador de programa é carregado no PC
  - Que programa está rodando ao lado?



Criação de Processos

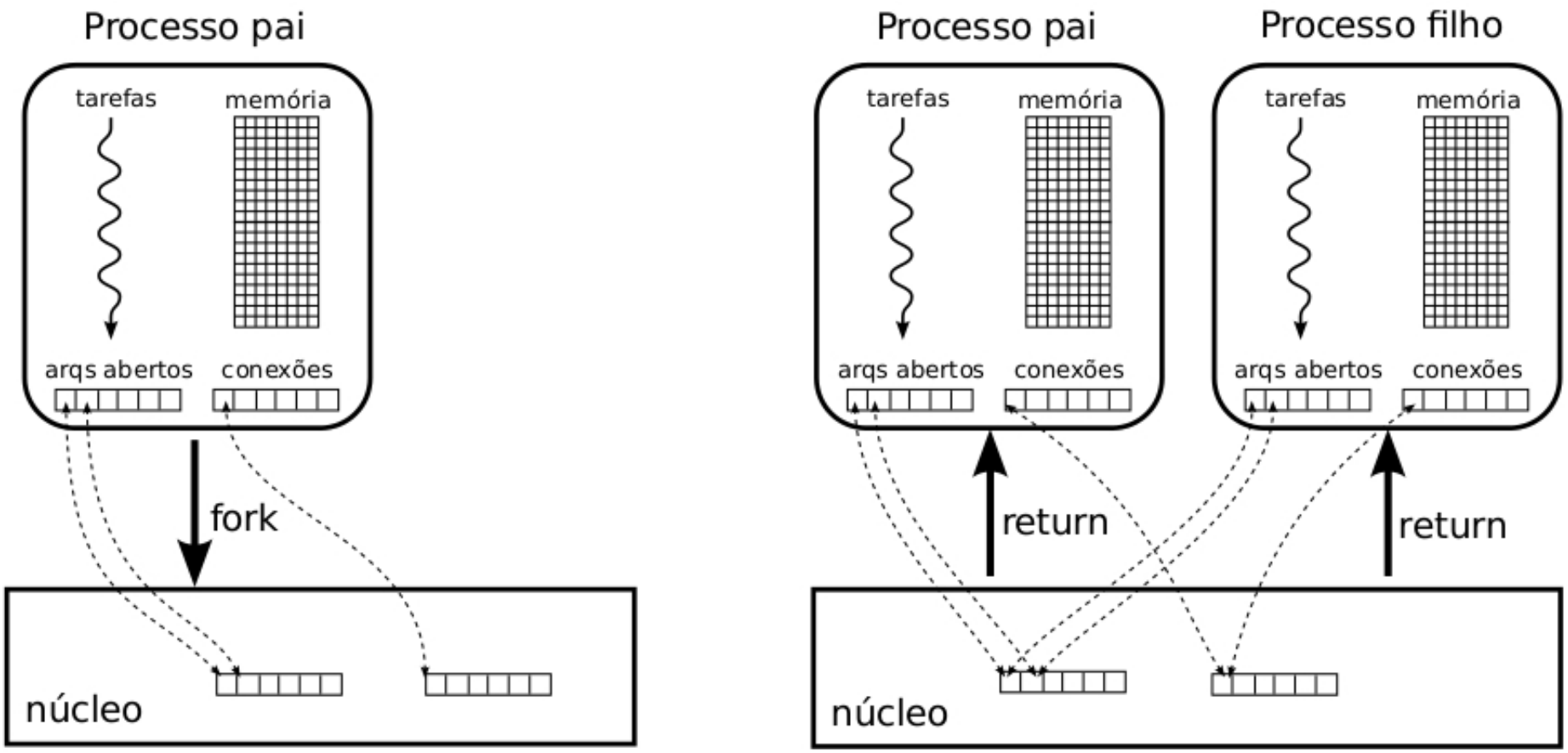
Processos precisam ser criados e finalizados a todo o momento:

- Inicialização do sistema;
- Execução de uma chamada de sistema para criação de processo, realizada por algum processo em execução;
- Requisição de usuário para criar um novo processo;
- Inicialização de um processo em batch
  - Mainframes com sistemas em batch;

Criação de Processos

- Processos são criados por outros processos, executando uma chamada ao sistema
  - UNIX: Fork
    - Cria clone do processo Pai: cópias exatas na memória, mas com identificadores diferentes
    - Depois o processo Filho executa um novo código, via execve, substituindo o conteúdo da memória
    - Permite que o filho execute algo enquanto ainda é clone (ex: manipular descritores de arquivo, redirecionando entrada e saída)
  - Windows: CreateProcess
    - Cria Filho, já carregando novo programa nele.

Criação de Processos – Fork



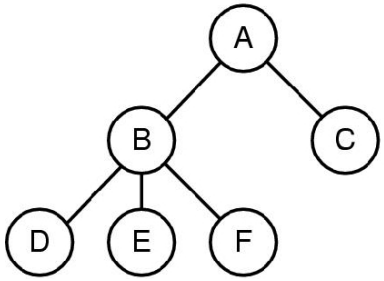
Embora fork seja chamada do pai, ela retorna tanto no pai quanto no filho

Hierarquia de Processos

- Unix: Pai cria processo filho, que cria neto
  - Possibilidade de Hierarquia
  - O comando pstree (linux) permite visualizar a árvore de processos do sistema
- Windows:
  - Todos os processos são iguais
  - Não existe o conceito de hierarquia
    - O Pai controla o filho via um identificador – Handle
    - Ele é livre para passar o identificador a outros processos, deserdando o Filho

Hierarquia de Processos

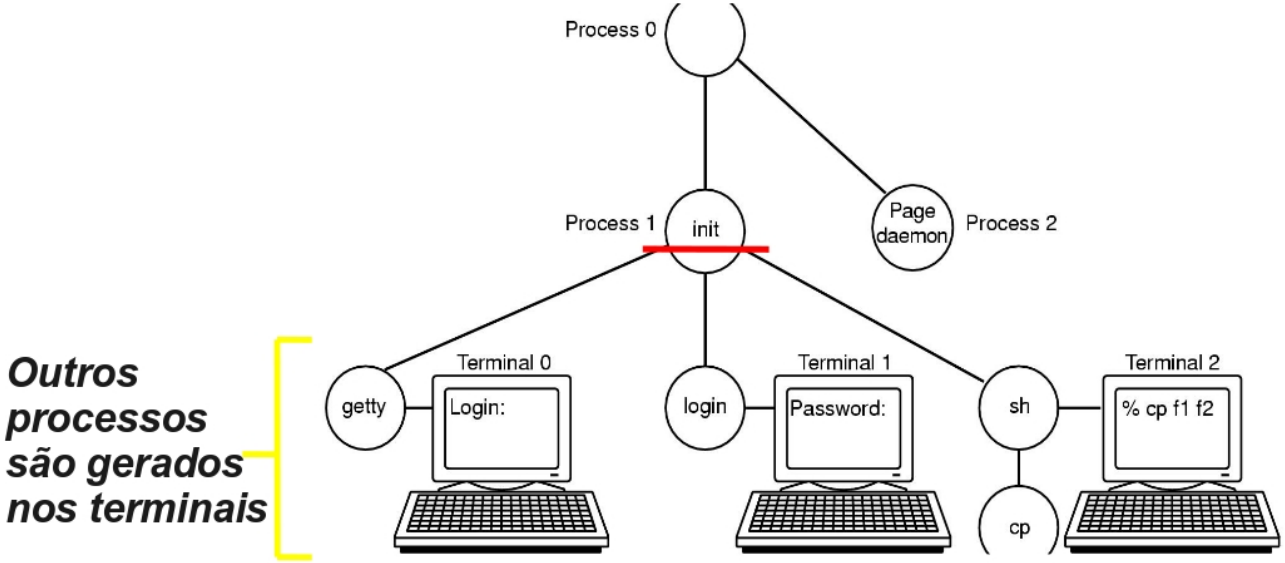
- Unix/Linux:
  - Forma grupos de processos (o processo e seus descendentes)
    - Ex: sinais do teclado são mandados aos membros do grupo associado com o teclado – normalmente todos os processos da janela atual



  - O filho não conhece o processo pai, mas o pai conhece o processo Filho → cabe ao pai abrir um canal de comunicação com o filho
    - O pai não pode passar adiante o identificador do filho → não pode deserdar o Filho

Hierarquia de Processos

- Unix/Linux:
  - Todo processo descende de init
    - init gera vários processos filhos para atender os vários terminais que existem no sistema;



Finalizando Processos – Condições

- Término normal (voluntário):
  - A tarefa a ser executada é finalizada;
  - Ao terminar, o processo executa uma chamada (comunicando ao SO que terminou): exit (UNIX) e ExitProcess (Windows)
- Término por erro (voluntário):
  - O processo sendo executado não pode ser finalizado
  - Ex: gcc filename.c, o arquivo filename.c não existe;

Finalizando Processos – Condições

- Término com erro fatal (involuntário);
  - Erro causado por algum erro no programa (bug)
  - Ex: Divisão por 0 (zero); Referência a memória inexistente; Execução de uma instrução ilegal;
  - Nesse caso, o processo é interrompido (via signal)
- Término (involuntário) causado por algum outro processo, via chamada a:
  - Kill (UNIX)
  - TerminateProcess (Windows)
  - O processo pedindo o término deve ter permissão para isso

Processos – Estados

- Um processo possui três estados básicos:
  - Executando: realmente usando a CPU naquele momento
  - Bloqueado: incapaz de executar enquanto um determinado evento externo não ocorrer
  - Pronto: em memória, pronto para executar (ou para continuar sua execução), apenas aguardando a disponibilidade do processador
  - Há também um quarto estado, problemático: zumbi
    - Processo que não deveria existir, mas que mantém cópia em memória
    - Ocorre quando o filho perde sua ligação com o pai

Processos – Estados

Zumbi

- Quando um programa filho termina antes do pai, o núcleo ainda mantém informação sobre ele
  - Como seu exit status, por exemplo
  - Uma entrada na tabela de processos (mais adiante)
- Ao chamar wait(), o pai recebe a informação e o núcleo descarta o processo
  - Nesse meio tempo, entre o término do filho e o wait, ele é um zumbi – se o pai travar aqui, há problemas...
  - Todo processo antes passa pelo estado de zumbi

Processos – Estados

Zumbi

- Mesmo que não rode, ele ocupa uma entrada na tabela
  - Que tem tamanho fixo (o número de processos é fixo)
  - Um grande número deles pode impedir a abertura de novos processos
- Se o pai termina sem chamar wait(), os filhos são adotados por init
  - Que cuida de limpá-los

Processos – Estados

Zumbi

- Mas se o pai não morre e não chama wait(), aí eles surgem
- Pode-se contudo matar manualmente um zumbi, com kill -18 PPID
  - Sinal mandado ao pai (PPID)
  - Alternativamente kill -s SIGCHLD ppid
  - Se o pai estiver programado para ignorar... aí só eliminando o pai

Processos – Estados

Zumbi

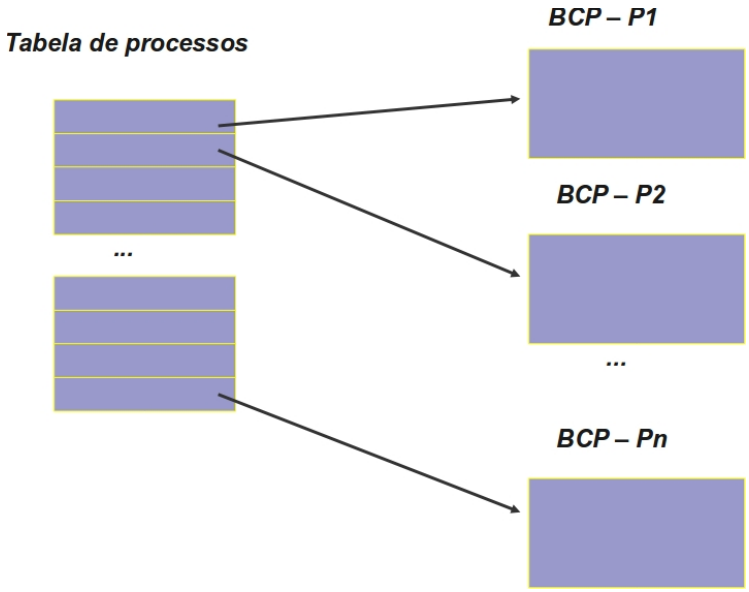
- Mas se o pai não morre e não chama wait(), aí eles surgem
- Pode-se contudo matar manualmente um zumbi, com kill -18 PPID
  - Sinal mandado ao pai (PPID)
  - Alternativamente kill -s SIGCHLD ppid
  - Se o pai estiver programado para ignorar... aí só eliminando o pai ... Sinistro!

Implementação de Processos

- Todo controle de processos é feito por uma tabela – Tabela de Processos
  - Os diversos processos em um computador são mantidos na Tabela de Processos
  - Cada processo constitui uma entrada na tabela
  - Cada entrada possui um ponteiro para o bloco de controle de processo (BCP) ou descritor de processo
    - BCP possui todas as informações do processo (contextos de hardware, software, endereço de memória – tudo que for necessário para se reiniciar um processo do ponto em que foi interrompido)
    - A tabela é então um arranjo ou lista ligada de ponteiros para BCPs

Implementação de Processos

- O BCP só não guarda o conteúdo do espaço de endereçamento do processo



- Assim, um processo é constituído de seu espaço de endereçamento e BCP (com seus registradores etc), representando uma entrada na tabela de processos

Implementação de Processos

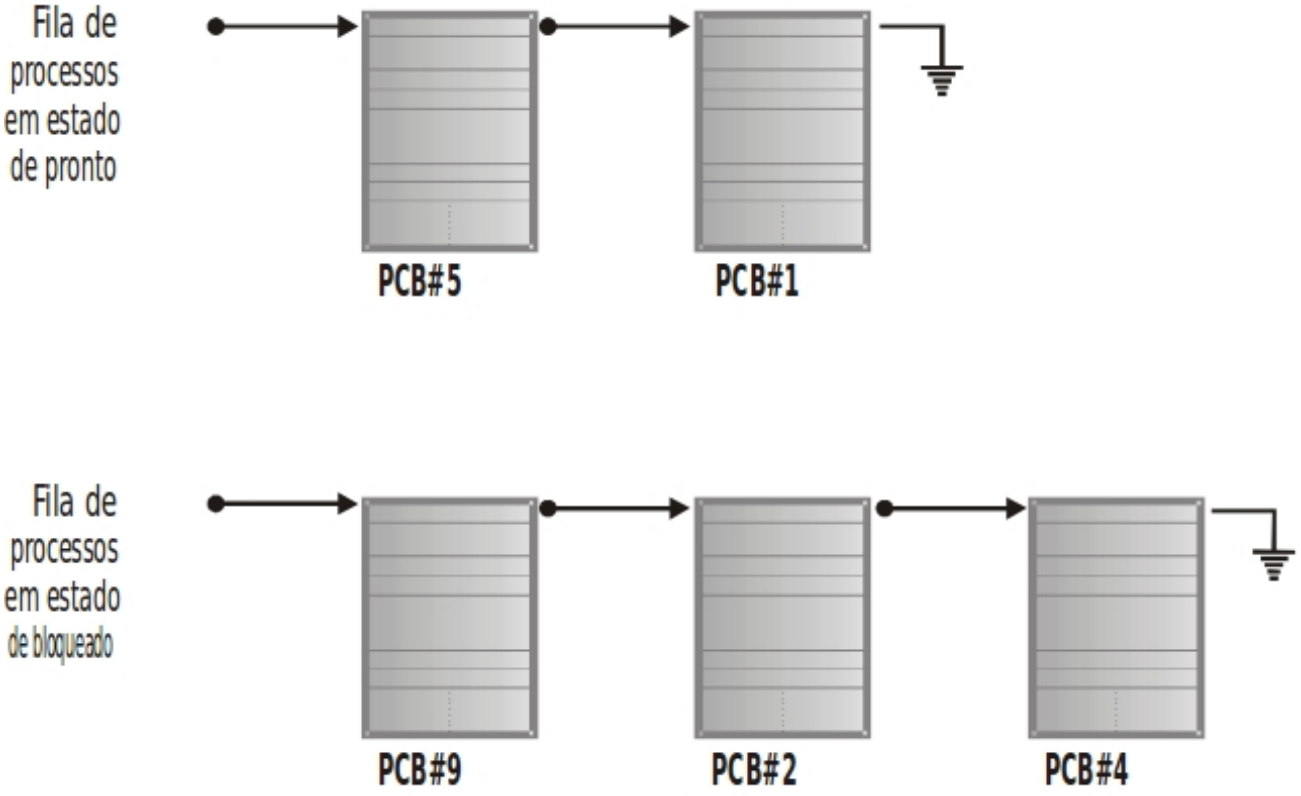
Process management	**Memory management**	**File management**
Registers	Pointer to text segment	Root directory
Program counter	Pointer to data segment	Working directory
Program status word	Pointer to stack segment	File descriptors
Stack pointer		User ID
Process state		Group ID
Priority		
Scheduling parameters		
Process ID		
Parent process		
Process group		
Signals		
Time when process started		
CPU time used		
Children's CPU time		
Time of next alarm		

Algumas informações do BCP

Etapas de Criação de Processos

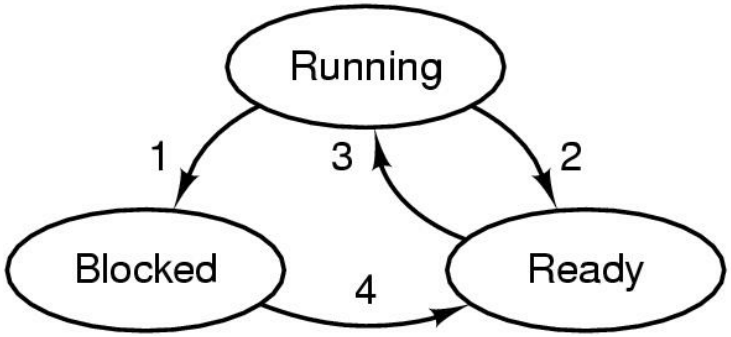
1. O SO atribui um identificador único (PID) ao processo
2. Aloca uma entrada na tabela de processos
3. Aloca espaço para o processo na memória
4. Inicializa o BCP
5. Coloca o endereço do BCP na fila apropriada (Pronto ou bloqueado)
6. Cria estruturas auxiliares

Criação de Processos: Filas



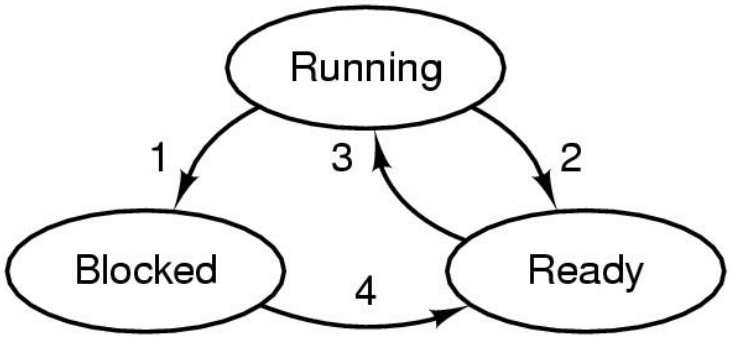
Mudança de Estado de Processos

- Balancear a carga da CPU requer trocar o processo que está rodando, dando lugar a outro que precise dela
  - Requer a mudança do estado do processo

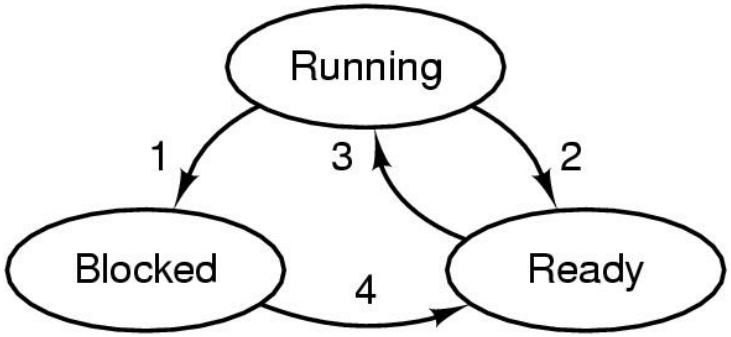


(1) O processo bloqueia aguardando entrada (E/S)
Um processo é bloqueado via (i) chamada ao sistema: block ou pause; ou (ii) se não há entrada disponível para que o processo continue sua execução

Mudança de Estado de Processos



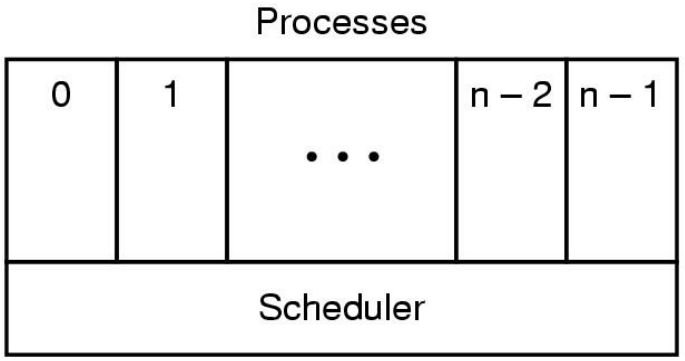
As transições 2 e 3 ocorrem durante o escalonamento de processos: o tempo destinado àquele processo acabou e outro processo é colocado no processador



A transição 4 é feita quando o evento esperado pelo processo bloqueado ocorre (Ex: E/S).
Se o processador está parado, o processo é executado imediatamente (3);
Se o processador está ocupado, o processo deve esperar sua vez

Escalonador

- A troca de processos, por sua vez, é feita pelo Escalonador de Processos
  - Processo que escolhe qual será o próximo processo a ser executado
    - Existem diversas técnicas/algoritmos para escalonamento de processos
    - Por vezes, feita diretamente pelo dispatcher (veremos mais adiante)
  - Nível mais baixo do SO



Escalonador

- O escalonamento é realizado com o auxílio do hardware
- O escalonador de Processos escolhe o processo que será executado pela CPU
  - Várias políticas possíveis
  - Veremos mais adiante...
- Junto com o dispatcher, contém todo o tratamento de interrupções, além de detalhes sobre iniciação e bloqueio de processos